N8N Website Health Check Workflow

Main Workflow (Runs every minute)

1. Schedule Trigger

- **Node**: Schedule Trigger
- **Interval**: Every 1 minute
- **Settings**: Use cron expression `* * * * *`

2. Get URLs Ready for Check

- **Node**: MySQL Query
- **Query**:

```sql
SELECT id, url, name, expected_response, timeout_seconds,
       failure_threshold, current_failure_count, priority,
       check_interval_minutes, expected_response_chars
FROM monitored_urls
WHERE is_active = TRUE
  AND (
    last_checked_at IS NULL OR
    last_checked_at <= DATE_SUB(NOW(), INTERVAL check_interval_minutes MINUTE)
  )
ORDER BY priority DESC, last_checked_at ASC
```

3. Split in Batches (If Many URLs)

- **Node**: Split in Batches
- **Batch Size**: 10 (to avoid overwhelming the system)

4. Process Each URL

- **Node**: Function
- **Purpose**: Prepare data for HTTP request
- **Code**:

```javascript
// Extract URL data
const urlData = $input.item.json;

return {
  json: {
    url_id: urlData.id,
    url: urlData.url,
    name: urlData.name,
    expected_response: urlData.expected_response,
    timeout: urlData.timeout_seconds * 1000, // Convert to milliseconds
    failure_threshold: urlData.failure_threshold,
    current_failures: urlData.current_failure_count,
    priority: urlData.priority,
    expected_chars: urlData.expected_response_chars || 200
  }
};
```

5. HTTP Request

- **Node**: HTTP Request
- **Method**: GET (or dynamic based on URL config)
- **URL**: `{{ $json.url }}`
- **Timeout**: `{{ $json.timeout }}`
- **Options**:
  - Follow redirects: Yes
  - Ignore SSL issues: No (for security)
  - Response format: String

6. Process Response

- **Node**: Function
- **Purpose**: Analyze response and prepare report data
- **Code**:

javascript

```
const startTime = Date.now();
const urlData = $('Process Each URL').item.json;

// Get response data
const response = $input.item;
let status = 'error';
let errorMessage = null;
let responseTime = null;
let httpStatusCode = null;
let responsePreview = null;

try {
  if (response.json) {
    // Successful response
    httpStatusCode = response.json.statusCode || 200;
    responseTime = response.json.responseTime || (Date.now() - startTime);

    // Check if status code is 200
    if (httpStatusCode === 200) {
      status = 'up';

      // Get response preview
      const responseBody = response.json.body || '';
      responsePreview = responseBody.substring(0, urlData.expected_chars);

      // Check expected response if defined
      if (urlData.expected_response &&
          !responseBody.includes(urlData.expected_response)) {
        status = 'error';
        errorMessage = 'Expected response string not found';
      }
    } else {
      status = 'down';
      errorMessage = `HTTP ${httpStatusCode}`;
    }
  } else {
    // Error in request
    status = 'error';
    errorMessage = response.error?.message || 'Unknown error';
  }
} catch (error) {
  status = 'error';
  errorMessage = error.message;
```

```
}

return {
  json: {
    url_id: urlData.url_id,
    url: urlData.url,
    name: urlData.name,
    status: status,
    response_time_ms: responseTime,
    http_status_code: httpStatusCode,
    response_body_preview: responsePreview,
    error_message: errorMessage,
    current_failures: urlData.current_failures,
    failure_threshold: urlData.failure_threshold,
    priority: urlData.priority,
    checked_at: new Date().toISOString()
  }
};
```

7. Save Health Report

- **Node**: MySQL Insert
- **Table**: health_reports
- **Query**:

```sql
INSERT INTO health_reports
(url_id, status, response_time_ms, http_status_code,
 response_body_preview, error_message, checked_at)
VALUES
({{ $json.url_id }}, '{{ $json.status }}', {{ $json.response_time_ms }},
 {{ $json.http_status_code }}, '{{ $json.response_body_preview }}',
 '{{ $json.error_message }}', '{{ $json.checked_at }}')
```

8. Update URL Status

- **Node**: MySQL Update
- **Purpose**: Update last check time and failure count
- **Query**:

```sql
UPDATE monitored_urls
SET
  last_checked_at = NOW(),
  last_status = '{{ $json.status }}',
  current_failure_count = CASE
    WHEN '{{ $json.status }}' = 'up' THEN 0
    ELSE current_failure_count + 1
  END
WHERE id = {{ $json.url_id }}
```

9. Check Alert Threshold

- **Node**: IF Condition
- **Condition**:

```javascript
// Check if we need to send alert
const data = $input.item.json;
const newFailureCount = data.status === 'up' ? 0 : data.current_failures + 1;

return newFailureCount >= data.failure_threshold && data.status !== 'up';
```

10. Send Alert (if needed)

- **Node**: Function
- **Purpose**: Prepare alert message
- **Code**:

```javascript
const data = $input.item.json;
const alertMessage = `
🚨 ALERT: Website Down
Name: ${data.name}
URL: ${data.url}
Status: ${data.status}
Error: ${data.error_message || 'Unknown'}
Priority: ${data.priority}
Time: ${new Date().toLocaleString()}
Consecutive Failures: ${data.current_failures + 1}
`;

return {
  json: {
    ...data,
    alert_message: alertMessage
  }
};
```

11. Get Notification Settings

- **Node**: MySQL Query
- **Query**:

```sql
SELECT notification_type, destination
FROM notification_settings
WHERE url_id = {{ $json.url_id }} AND is_active = TRUE
```

12. Send Email Alert

- **Node**: Send Email
- **To**: `{{ $json.destination }}`
- **Subject**: `🚨 Website Alert: {{ $('Process Response').item.json.name }}`
- **Body**: `{{ $('Send Alert').item.json.alert_message }}`

Additional Workflows

Cleanup Workflow (Daily)

- **Trigger**: Schedule (daily at 2 AM)
- **Node**: MySQL Query
- **Query**: `CALL CleanOldHealthReports()`

Recovery Alert Workflow

- **Purpose**: Send "site is back up" notifications
- **Trigger**: When status changes from down to up
- **Implementation**: Add to main flow after status update

Configuration Tips

1. Error Handling

- Add error handling nodes after HTTP requests
- Set up retry logic for failed requests
- Log errors to separate table for debugging

2. Performance Optimization

- Use connection pooling for MySQL
- Implement batch processing for many URLs
- Add delays between requests to avoid overwhelming servers

3. Monitoring the Monitor

- Create a dashboard view of the health check system itself
- Set up alerts if the n8n workflow fails
- Monitor database performance and cleanup

4. Notification Features

- Implement notification cooldown (don't spam alerts)
- Add different alert levels based on priority
- Support for escalation (alert different people based on time)

Usage Examples

Adding a New URL to Monitor:

```sql
INSERT INTO monitored_urls (url, name, description, priority, check_interval_minutes,
VALUES ('https://mysite.com', 'My Website', 'Company main site', 'critical', 1, 'admin
```

Setting up Email Notifications:

```sql
INSERT INTO notification_settings (url_id, notification_type, destination)
VALUES (1, 'email', 'admin@company.com');
```

Setting up SMS Notifications:

```sql
INSERT INTO notification_settings (url_id, notification_type, destination)
VALUES (1, 'sms', '1234567890@vztext.com');
```

Viewing Current Status:

```sql
SELECT * FROM url_status_summary;
```